

Linear regression by using Deep Neural network: Implement Boston housing price prediction problem by Linear regression using Deep Neural network. Use Boston House price prediction dataset.

Step 1: Import required libraries

```
import numpy as np          # For numerical operations
import pandas as pd         # For data handling (CSV, tables)
import matplotlib.pyplot as plt  # For plotting graphs

from sklearn.model_selection import train_test_split # To split dataset
from sklearn.preprocessing import StandardScaler    # To normalize data

import tensorflow as tf

from tensorflow.keras.models import Sequential      # Model type
from tensorflow.keras.layers import Dense          # Fully connected layer
```

Step 2: Load Dataset

```
df = pd.read_csv("1_boston_housing.csv") # Load CSV file into dataframe

# Remove extra quotes from column names (important for this dataset)
df.columns = df.columns.str.replace("'", "")

df.head() # Display first 5 rows (for understanding data)
```

```

# Step 3: Split Features (X) and Target (y)

X = df.drop("MEDV", axis=1) # All columns except MEDV → input features
y = df["MEDV"]             # MEDV → house price (target/output)

print(X.shape, y.shape)    # Print dimensions of data


# Step 4: Train-Test Split
# 80% data for training, 20% for testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)


# Step 5: Feature Scaling (Normalization)
# Important because neural networks work better with scaled data
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train) # Fit + transform training data
X_test = scaler.transform(X_test)       # Only transform test data


# Initial model (simple linear regression using neural network)
model = Sequential()

# Dense layer with 1 neuron → acts like linear regression
model.add(Dense(1, input_shape=(X_train.shape[1],), activation='linear'))

# Compile model

```

```
model.compile(  
    optimizer='adam', # Optimization algorithm  
    loss='mse',      # Mean Squared Error (for regression)  
    metrics=['mae']   # Mean Absolute Error  
)
```

```
# Train model
```

```
history = model.fit(  
    X_train, y_train,  
    epochs=100,      # Number of iterations  
    validation_split=0.05, # 5% data for validation  
    verbose=1        # Show training progress  
)
```

```
# Step 6: Build Deep Neural Network (again defined)
```

```
model = Sequential()
```

```
# Again single neuron (linear regression behavior)
```

```
model.add(Dense(1, input_shape=(X_train.shape[1],), activation='linear'))
```

```
# Compile model
```

```
model.compile(  
    optimizer='adam',  
    loss='mse',  
    metrics=['mae']  
)
```

```
model.summary() # Show model architecture
```

Step 7: Train Model

```
history = model.fit(  
    X_train, y_train,  
    epochs=100,  
    batch_size=16,      # Number of samples per batch  
    validation_split=0.2, # 20% validation data  
    verbose=1  
)
```

Step 8: Evaluate Model

```
loss, mae = model.evaluate(X_test, y_test)  
  
print("Test Loss (MSE):", loss) # Error (squared)  
print("Test MAE:", mae)        # Absolute error
```

Step 9: Predictions

```
y_pred = model.predict(X_test)
```

Compare actual vs predicted values

```
for i in range(5):  
    print(f"Actual: {y_test.iloc[i]:.2f} | Predicted: {y_pred[i][0]:.2f}")
```

Step 10: Plot Training Graph

```
plt.plot(history.history['loss'], label='Training Loss') # Training loss
```

```
plt.plot(history.history['val_loss'], label='Validation Loss') # Validation loss
```

```
plt.legend()
```

```
plt.title("Loss Graph")
```

```
plt.xlabel("Epochs")
```

```
plt.ylabel("Loss")
```

```
plt.show()
```

Line-by-Line Explanation (Easy + Viva Ready)

◆ Step 1: Import Libraries

```
import numpy as np
```

👉 Used for numerical operations like arrays and calculations.

```
import pandas as pd
```

👉 Used to load and handle dataset (CSV file).

```
import matplotlib.pyplot as plt
```

👉 Used to plot graphs (loss curve).

```
from sklearn.model_selection import train_test_split
```

👉 Used to split data into training and testing sets.

```
from sklearn.preprocessing import StandardScaler
```

👉 Used to normalize/scale data (important for neural networks).

```
import tensorflow as tf
```

👉 TensorFlow is the main deep learning library.

```
from tensorflow.keras.models import Sequential
```

👉 Sequential model means layers are added one after another.

```
from tensorflow.keras.layers import Dense
```

👉 Dense layer = fully connected layer (basic building block of neural network).

◆ Step 2: Load Dataset

```
df = pd.read_csv("1_boston_housing.csv")
```

👉 Loads dataset into a dataframe.

```
df.columns = df.columns.str.replace("'", "")
```

👉 Removes unwanted quotes from column names.

```
df.head()
```

👉 Displays first 5 rows to understand data.

◆ Step 3: Split Features and Target

```
X = df.drop("MEDV", axis=1)
```

👉 Input features (all columns except MEDV).

```
y = df["MEDV"]
```

👉 Target/output → house price.

```
print(X.shape, y.shape)
```

👉 Shows number of rows and columns.

◆ Step 4: Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

👉 Splits data:

- 80% → Training
 - 20% → Testing
- 👉 `random_state=42` ensures same split every time.
-

◆ Step 5: Feature Scaling

```
scaler = StandardScaler()
```

👉 Creates scaler object.

```
X_train = scaler.fit_transform(X_train)
```

👉 Learns mean & std and scales training data.

```
X_test = scaler.transform(X_test)
```

👉 Applies same scaling to test data.

📌 Why scaling?

Neural networks perform better when data is normalized.

◆ Step 6: Build Model (Neural Network)

```
model = Sequential()
```

👉 Creates empty neural network.

```
model.add(Dense(1, input_shape=(X_train.shape[1],), activation='linear'))
```

👉 Adds layer:

- 1 neuron → output value
- input_shape = number of features
- linear activation → used for regression

📌 This behaves like **Linear Regression**

◆ Compile Model

```
model.compile(  
    optimizer='adam',  
    loss='mse',  
    metrics=['mae']  
)
```

👉 Defines:

- **optimizer (adam)** → improves learning
 - **loss (mse)** → error calculation
 - **mae** → evaluation metric
-

◆ Train Model

```
history = model.fit(  
    X_train, y_train,  
    epochs=100,  
    validation_split=0.05,  
    verbose=1  
)
```

👉 Model learns from data:

- epochs = number of iterations
- validation_split = 5% data for validation
- verbose = shows training progress

◆ Step 6 Again (Redundant Model)

```
model = Sequential()
```

👉 New model is created again (overwrites previous one).

```
model.add(Dense(1, input_shape=(X_train.shape[1],), activation='linear'))
```

👉 Same single neuron.

```
model.compile(...)
```

👉 Same settings again.

```
model.summary()
```

👉 Shows model structure.

📌 ⚠️ IMPORTANT:

👉 This part is **duplicate** (you can say this in viva if asked).

◆ Step 7: Train Again

```
history = model.fit(
    X_train, y_train,
    epochs=100,
    batch_size=16,
    validation_split=0.2,
    verbose=1
)
```

👉 Training again with:

- `batch_size = 16` samples at a time
 - `validation_split = 20%`
-

◆ Step 8: Evaluate Model

```
loss, mae = model.evaluate(X_test, y_test)
```

👉 Tests model on unseen data.

```
print("Test Loss (MSE):", loss)
```

```
print("Test MAE:", mae)
```

👉 Displays error values.

◆ Step 9: Predictions

```
y_pred = model.predict(X_test)
```

👉 Predict house prices.

```
for i in range(5):  
    print(f'Actual: {y_test.iloc[i]:.2f} | Predicted: {y_pred[i][0]:.2f}')
```

👉 Compares real vs predicted values.

◆ Step 10: Plot Graph

```
plt.plot(history.history['loss'], label='Training Loss')
```

👉 Training loss graph.

```
plt.plot(history.history['val_loss'], label='Validation Loss')
```

👉 Validation loss graph.

```
plt.legend()  
plt.title("Loss Graph")  
plt.xlabel("Epochs")  
plt.ylabel("Loss")  
plt.show()
```

👉 Displays graph.

🎯 Final Viva Summary (Say This Confidently)

👉 “This project implements Boston Housing Price Prediction using a Neural Network.

The model uses a single dense neuron, which behaves like linear regression. Data is preprocessed using scaling, then trained using Adam optimizer and MSE loss. Finally, model performance is evaluated using MAE and visualized using loss graphs.”

⚠️ Smart Point (Impress Examiner)

👉 “Although this is implemented using a neural network, since it has only one neuron and no hidden layers, it is equivalent to linear regression. It can be extended to deep learning by adding hidden layers.”

🧠 Dry Run of Your Code (Simple + Examiner Friendly)

◆ Step 1: Program Starts

👉 Libraries are loaded
(No output yet)

◆ Step 2: Dataset Load

```
df = pd.read_csv("1_boston_housing.csv")
```

👉 Suppose dataset looks like this:

CRIM RM AGE MEDV

0.1 6.5 65 24

0.2 7.0 70 30

0.3 6.0 80 22

👉 Data stored in **df (DataFrame)**

◆ Step 3: Split X and y

```
X = df.drop("MEDV", axis=1)  
y = df["MEDV"]
```

👉 Now:

X (inputs):

CRIM RM AGE

0.1 6.5 65

0.2 7.0 70

y (output):

MEDV

24

30

◆ Step 4: Train-Test Split

`X_train, X_test, y_train, y_test = train_test_split(...)`

👉 Example split:

- `X_train` → 80% data
- `X_test` → 20% data

👉 Suppose:

Training Data:

`X_train` → 2 rows

`y_train` → 2 values

Testing Data:

`X_test` → 1 row

`y_test` → 1 value

◆ Step 5: Scaling

`scaler.fit_transform(X_train)`

👉 Converts data like:

RM

6.5

7.0

→ into normalized values:

RM_scaled

-0.5

0.5

✚ Mean = 0, Std = 1

◆ Step 6: Model Creation

```
model = Sequential()  
model.add(Dense(1, input_shape=(3,), activation='linear'))
```

👉 Model structure:

Input (3 features) → [Neuron] → Output (1 value)

✚ This is **linear regression**

◆ Step 7: Training

```
model.fit(X_train, y_train, epochs=100)
```

👉 What happens internally:

1. Model takes input (X_train)
2. Applies weights:

$$y = w1*x1 + w2*x2 + w3*x3 + b$$

3. Predicts output
4. Compares with actual (y_train)
5. Calculates error (MSE)
6. Updates weights using Adam optimizer

🔄 Repeats for 100 epochs

◆ Step 8: After Training

👉 Model learns best weights:

Example:

w1 = 2.1
w2 = 5.3
w3 = -1.2
b = 10

◆ Step 9: Testing

`model.evaluate(X_test, y_test)`

👉 Model predicts value for unseen data

Example:

Actual = 25

Predicted = 23.5

👉 Error calculated:

- MSE
 - MAE
-

◆ Step 10: Prediction

`y_pred = model.predict(X_test)`

👉 Example output:

[23.5, 28.2, 19.7]

◆ Step 11: Comparison

Actual: 25 | Predicted: 23.5

Actual: 30 | Predicted: 28.2

👉 Shows model accuracy

◆ Step 12: Graph

`plt.plot(history.history['loss'])`

👉 Graph shows:

- Loss ↓ decreasing → good learning
- If loss ↑ → bad model

Final Dry Run Summary (Say This in Viva)



“First, dataset is loaded and split into features and target.
Then data is divided into training and testing.
Training data is normalized using StandardScaler.
A neural network with one neuron is created, which behaves like linear regression.
The model learns weights by minimizing error using Adam optimizer.
Finally, the model is tested on unseen data and predictions are compared with actual values.”

Critical Insight (Examiner Trap)

 If examiner asks:

 “*Where is deep learning here?*”

 Answer:

“Currently, there is no hidden layer, so it behaves like linear regression. Deep learning can be achieved by adding hidden layers.”

Top Viva Questions + Smart Answers (Boston Housing DNN)

◆ 1. What is your project about?



“This project predicts house prices using a neural network. We use the Boston Housing dataset, where input features like crime rate, number of rooms, etc., are used to predict the median house price (MEDV).”

◆ 2. What type of problem is this?



“This is a **regression problem** because the output is a continuous value (house price).”

◆ 3. What is the Boston Housing dataset?



“It is a dataset containing housing details of Boston suburbs, with features like crime rate, number of rooms, age of houses, etc., and the target variable MEDV representing house price.”

◆ 4. Why did you use a Neural Network?



“Neural networks can model complex relationships between features and output. Although in this implementation we used a single neuron, it can be extended to deep learning by adding hidden layers.”

◆ 5. Why only one neuron?



“A single neuron with linear activation behaves like **linear regression**. This is a basic implementation, and deeper models can be created by adding more layers.”

◆ 6. What is a Dense layer?



“A Dense layer is a fully connected layer where each neuron is connected to all inputs from the previous layer.”

◆ 7. What is activation function?



“It defines how the neuron processes input.
In this case, we used **linear activation** because it is a regression problem.”

◆ 8. Why not use ReLU?



“ReLU is mainly used in hidden layers.
For output layer in regression, linear activation is preferred to produce continuous values.”

◆ 9. What is loss function?



“It measures the error between predicted and actual values.
We used **Mean Squared Error (MSE)** for regression.”

◆ 10. What is optimizer?



“Optimizer updates model weights to minimize loss.
We used **Adam optimizer**, which is efficient and widely used.”

◆ 11. What is MAE?



“Mean Absolute Error measures the average absolute difference between predicted and actual values.”

◆ 12. Why scaling is required?



“Feature scaling ensures all inputs are on the same scale, which helps the neural network converge faster and improves accuracy.”

◆ 13. Difference between fit and transform?



- **fit()** → learns parameters (mean, std)
- **transform()** → applies scaling



“We use fit_transform on training data and only transform on test data.”

◆ 14. What is train-test split?



“It divides data into training and testing sets.

Training data is used to learn, and testing data is used to evaluate performance.”

◆ 15. What is epoch?



“One complete pass of training data through the model.”

◆ 16. What is batch size?



“Number of samples processed before updating weights.”

◆ 17. What is overfitting?



“When model performs well on training data but poorly on new data.”

◆ 18. How to reduce overfitting?



- Use more data

- Add regularization
 - Reduce model complexity
 - Use dropout
-

◆ **19. What is underfitting?**



“When model is too simple and cannot learn patterns properly.”

◆ **20. What is validation split?**



“A portion of training data used to validate model during training.”

◆ **21. What is Sequential model?**



“A linear stack of layers where output of one layer is input to next.”

◆ **22. What is backpropagation?**



“It is the process of updating weights by calculating gradients of loss function.”

◆ **23. What is gradient descent?**



“An optimization technique to minimize loss by updating weights step by step.”

◆ **24. Why use Adam optimizer?**



“Adam combines advantages of momentum and RMSProp, making it faster and efficient.”

◆ **25. What is the role of weights and bias?**



- Weights → importance of each feature
 - Bias → shifts output
-

♦ 26. What happens during training?



“Model predicts output → compares with actual → calculates error → updates weights.”

♦ 27. Why use MSE instead of MAE?



“MSE penalizes larger errors more strongly, making it suitable for regression.”

♦ 28. What is the output of your model?



“A single numeric value representing predicted house price.”

♦ 29. How can you improve this model?



- Add hidden layers
 - Use ReLU activation
 - Tune hyperparameters
 - Use more data
-

♦ 30. Is this Deep Learning?



“Technically no, because there are no hidden layers.
It is a neural network implementation of linear regression.”



Bonus: Killer Answer (Say This to Impress Examiner)



“Although this model uses a neural network framework, since it contains only a single neuron and no hidden layers, it behaves like linear regression. However, it provides flexibility to extend into a deep neural network by adding hidden layers and non-linear activation functions.”